CS3400: Data Structures & Algorithms

Hospital Management System

Mini Project — Group Planning Document

Group Project · C++ · 3 Students

10	3	5	100
Modules	Students	Phases	Total Marks

1. What This Project Is

CS3400 requires each group to build a **campus/institution management system in C++** that demonstrates every major data structure and algorithm covered in the course. The actual system domain (hospital, library, transport, etc.) is just the wrapper — **demonstrating understanding of the DS & algorithms is the real objective.**

Our group chose the **Hospital Management System** domain, managing three resource types: Patients, Wards, and Equipment.

Assessment Breakdown

Criterion	Marks
Problem understanding and system design	10
Appropriate use of data structures	20
Algorithms — searching and sorting	15
Trees, hashing and priority management	15
Graph implementation and traversal	10
Code quality and organisation	10
Complexity analysis	5
Testing and robustness	5
Documentation	5
Demonstration and individual understanding	5
TOTAL	**100**

2. The 10 Modules

Each module maps directly to a data structure or algorithm. The table below shows what we use and the hospital context it fits into.

#	Module	DS / Algorithm	Hospital Context
1	Resource Management	Classes / Arrays	Manage patients, wards, equipment
2	Dynamic Storage	Custom Linked List	Patient records list
3	Request Queue	Queue (FIFO)	Walk-in admission queue
4	Undo Operations	Stack (LIFO)	Undo discharge / record update
5	Priority Requests	Heap / Priority Queue	Triage: Emergency → Normal
6	Search & Sort	Linear, Binary, Merge, Bubble	Search/sort patients, wards
7	Tree Organisation	BST	Patient records by ID
8	Fast Lookup	Hash Table	PatientID → instant record
9	Campus Navigation	Graph + BFS + DFS	Hospital department map
10	Data Persistence	File I/O	Load/save .txt files

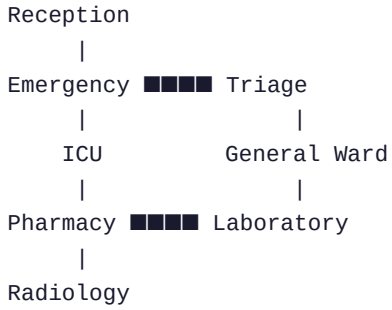
Key Point: No STL for Core Structures

The project explicitly requires students to **implement core data structures themselves** rather than using std::list, std::queue, std::stack, etc. 20 of the 100 marks are for correct DS implementation — this is where most marks are lost.

3. The Three Resources

Patient	Ward	Equipment
PatientID	WardID	EquipmentID
Name	Name	Name
Age	Type	Type
Condition	Capacity	Location
AssignedWard	CurrentOccupancy	Status
AdmitDate	Status	LastServiced
Status		
Priority		

Hospital Graph — Department Connections



Nodes = departments. Edges = corridors/pathways. BFS finds all reachable departments. DFS does full traversal. Dijkstra's finds the shortest route between two departments.

4. Project File Structure

```
CS3400_Hospital/
■
■■■ main.cpp
■
■■■ Patient.h / Patient.cpp
■■■ Ward.h / Ward.cpp
■■■ Equipment.h / Equipment.cpp
■
■■■ LinkedList.h / LinkedList.cpp        ← patient records
■■■ Queue.h / Queue.cpp                  ← admission queue
■■■ Stack.h / Stack.cpp                  ← undo operations
■■■ PriorityQueue.h / PriorityQueue.cpp  ← triage heap
■■■ BST.h / BST.cpp                      ← patient BST by ID
■■■ HashTable.h / HashTable.cpp          ← fast lookup
■■■ Graph.h / Graph.cpp                  ← hospital map
■■■ Sorting.h / Sorting.cpp              ← sort algorithms
■■■ FileManager.h / FileManager.cpp      ← file I/O
■
■■■ data/
    ■■■ patients.txt
    ■■■ wards.txt
    ■■■ equipment.txt
    ■■■ requests.txt
    ■■■ locations.txt
```

5. System Menu

```
========================================
   CS3400 HOSPITAL MANAGEMENT SYSTEM
========================================
 1.  Patient Management
 2.  Ward Management
 3.  Equipment Management
 4.  Admission Queue
 5.  Triage System (Priority Queue)
 6.  Undo Last Operation
 7.  Search & Sort
 8.  Patient BST
 9.  Resource Lookup (Hash Table)
10.  Hospital Navigation (Graph)
11.  System Analysis & Complexity
 0.  Exit
========================================
```

6. Work Split — By Student

Student A — Core Data & Resources

Task	Detail
3 Resource structs	Define Patient, Ward, Equipment classes with all attributes
Custom LinkedList	Insert front/end/pos, delete, traverse — NO std::list
CRUD for all 3 resources	Add, remove, update, display for patients/wards/equipment
Linear search	Search across all 3 resource types
Binary search	Binary search on sorted patient list
Sorting	Bubble/Insertion sort + Merge/Quick sort — at least one O(n log n)
File I/O	Load all data files on startup, save all on exit

Student B — Requests & Operations

Task	Detail
Main menu skeleton	All 10 menu options as stubs in Phase 1
Admission Queue (FIFO)	Walk-in patients wait in order — custom queue, no std::queue
Triage Priority Queue	Heap-based: Emergency=1, Critical=2, Urgent=3, Normal=4
Undo Stack	Undo discharge, undo record update, undo ward assignment
Request processing	Dequeue admission → assign to ward → update occupancy
Priority processing	Pull highest priority patient from triage heap first

Student C — Advanced DS & Navigation

Task	Detail
BST	Insert, search, delete patients by PatientID
BST traversals	Inorder (sorted IDs), Preorder, Level-order — at least 2
Hash Table	PatientID/EquipmentID → record, custom hash fn, collision handling
Hospital Graph	All departments as nodes, corridors as edges
BFS	Find all reachable departments from a starting point
DFS	Full hospital traversal
Shortest path	Dijkstra's algorithm between two departments

7. Phase-by-Phase Student Jobs

Phase 1 — Foundation

Student A

→ Define Patient, Ward, Equipment structs/classes with all attributes
→ Implement custom LinkedList for patients (insert front/end/pos, delete, traverse)
→ Basic CRUD menu for all 3 resources (add, remove, update, display all)
→ Test that linked list works correctly before anyone else touches it

Student B

→ Set up the main menu skeleton with all 10 options as stubs
→ Define Request struct (RequestID, patient info, type, time, status)
→ Nothing else yet — focus on getting the menu shell working so everyone has something to plug into

Student C

→ Plan the hospital graph on paper (decide all nodes and edges)
→ Plan the hash table size and hash function on paper
→ Nothing to code yet — use this phase to design so Phase 4 is smooth

Phase 2 — Requests

Student A

→ Nothing new to build — test and fix Phase 1 code
→ Make sure LinkedList is solid because B and C will depend on it soon

Student B

→ Implement Queue (custom, FIFO) for walk-in admissions
→ Implement Stack (custom) for undo operations
→ Connect undo stack to at least 2 operations (undo discharge, undo record update)
→ Implement PriorityQueue (heap-based) for triage
→ Demo: enqueue patients → dequeue in order → show undo working

Student C

→ Nothing new to code yet
→ Help B test the queue and stack
→ Finalise graph design and hash function on paper

Phase 3 — Organisation

Student A

→ Implement linear search across all 3 resource types
→ Implement binary search on sorted patient list
→ Implement 2 sorting algorithms (e.g. insertion sort + merge sort)

→ Sorting criteria: patient ID, name, admission date, condition priority

→ Compare the two sort algorithms (speed, complexity)

Student B

→ Connect admission queue to ward assignment — when patient dequeued → assign to available ward → update ward occupancy

→ Connect triage priority queue to patient status updates

→ Make sure undo stack tracks ward assignments too

Student C

→ Implement BST with patient records organised by PatientID

→ Insert, search, delete on BST

→ Implement inorder, preorder, and level-order traversals

→ Test BST independently before integration

Phase 4 — Advanced

Student A

→ Start file I/O — write save/load for patients.txt, wards.txt, equipment.txt

→ Test loading sample data on startup

Student B

→ Help test and fix any queue/stack/heap bugs that surface during integration

→ Write complexity analysis for queue enqueue/dequeue, heap insert, stack push/pop

Student C

→ Implement HashTable — custom hash function, collision handling (separate chaining recommended)

→ Support lookup by PatientID and EquipmentID

→ Implement Graph with all hospital departments as nodes

→ Implement BFS and DFS on the graph

→ Implement Dijkstra's for shortest path between departments

Phase 5 — Integration & Analysis

Student A

→ Finalise and test file I/O for all data files

→ Make sure data loads correctly into linked list on startup

→ Write complexity analysis section of report (search & sort operations)

Student B

→ Write complexity analysis for queue, stack, heap operations

→ Test all edge cases: empty queue, empty stack, full ward, duplicate patient ID

→ Write the testing section of the report

Student C

→ Connect BST and hash table to the same patient data A manages

→ Test graph traversals with the full hospital map

→ Write complexity analysis for BST, hash table, graph operations

8. Complexity Reference (Required Analysis)

The project requires complexity analysis of at least 5 operations. Each student should own the analysis for their modules.

Operation	Data Structure	Best	Average	Worst	Who
Enqueue patient	Queue	O(1)	O(1)	O(1)	B
Dequeue patient	Queue	O(1)	O(1)	O(1)	B
Push undo	Stack	O(1)	O(1)	O(1)	B
Triage insert	Heap	O(1)	O(log n)	O(log n)	B
Triage extract-max	Heap	O(log n)	O(log n)	O(log n)	B
Linear search	Linked List	O(1)	O(n)	O(n)	A
Binary search	Sorted Array	O(1)	O(log n)	O(log n)	A
Bubble/Insertion sort	Array	O(n)	O(n^2)	O(n^2)	A
Merge/Quick sort	Array	O(n log n)	O(n log n)	O(n log n)	A
BST insert/search	BST	O(1)	O(log n)	O(n)	C
Hash table lookup	Hash Table	O(1)	O(1)	O(n)	C
BFS traversal	Graph	O(V+E)	O(V+E)	O(V+E)	C
DFS traversal	Graph	O(V+E)	O(V+E)	O(V+E)	C
Dijkstra's shortest	Graph + Heap	O(E log V)	O(E log V)	O(E log V)	C

9. Critical Sync Points

These are the two moments where team members must coordinate before anyone moves forward:

Checkpoint	When	What Must Be Done	Who Waits
Checkpoint 1	End of Phase 1	Patient, Ward, Equipment structs must be finalised and stable.	B and C wait for A to build their module
Checkpoint 2	End of Phase 3	The patient data structure must be stable before C builds the BST and hash table on top of it.	C waits for A

Important: Every student must understand the complete system. During the demo, any group member can be asked about any part — a student who cannot explain code they didn't write may receive a lower individual mark.